Emploid Stripe Subscriptions & Billing

1. Billing Data Model

To monetize advanced job tracking features and assistant options, Emploid provides a **Pro Tier** subscription. This tier is tracked directly within the PostgreSQL `public.users` table:

- **`is_pro` (boolean):** Identifies whether the user has an active Pro subscription (default `false`).
- **`stripe_customer_id` (text):** Stores the unique identifier created by Stripe for billing association.

---

2. Checkout Flow

The frontend redirects users to Stripe to initiate payment through a dedicated API endpoint.

Create Checkout Endpoint (`/api/stripe/create-checkout/route.ts`)

- **Stripe Client Initialization:** Instantiates the Stripe library using `STRIPE_SECRET_KEY` from environment variables.
- **Price Verification:** Reads the subscription plan configuration from `STRIPE_PRO_PRICE_ID`. If this price ID is missing, it returns a 503 configuration error.
- **User Validation:** Accesses the active user's session.
- **Customer Lookup and Creation:**

1. Queries the database table `public.users` using the authenticated user ID. 2. If `stripe_customer_id` is present, it uses the existing ID. 3. If missing, it calls `stripe.customers.create({ email, name })` to register a new customer in Stripe. 4. Persists the new `stripe_customer_id` back to the user's database record.

- **Checkout Session Setup:** Calls `stripe.checkout.sessions.create()` with:
- `customer`: The resolved customer ID.
- `payment_method_types`: Defaults to standard payment methods.
- `line_items`: A single subscription item mapped to `STRIPE_PRO_PRICE_ID`.
- `mode`: `'subscription'`.
- `success_url`: Redirects the user to `/tracker?session_id={CHECKOUT_SESSION_ID}`.
- `cancel_url`: Redirects the user to `/tracker` (main dashboard).
- `metadata`: Attaches the user's database ID (`userId: user.id`) to link the transaction back on checkout completion.
- **Response:** Returns the checkout URL `NextResponse.json({ url: session.url })` for the client to redirect the browser.

---

3. Stripe Webhook Integration

To handle asynchronous subscription changes (successful checkouts, renewals, and cancellations), Emploid exposes a webhook endpoint.

Webhook Listener (`/api/stripe/webhook/route.ts`)

- **Signature Verification:** Reads the `stripe-signature` header. Calls `stripe.webhooks.constructEvent()` with the raw request body, signature, and `STRIPE_WEBHOOK_SECRET` to verify that the request originated from Stripe and was not tampered with.
- **Event Handling:**

Event 1: `checkout.session.completed` Fires when a user completes the payment flow.

- **Action:** Extends user access.
- **Flow:**

1. Extracts `metadata.userId` and the Stripe `customer` ID from the event object. 2. Executes a database write to the `users` table:

  - Sets `is_pro = true`.
  - Updates `stripe_customer_id = session.customer` (if not already set).

3. Logs the upgrade confirmation.

Event 2: `customer.subscription.deleted` Fires when a user's subscription expires, is cancelled, or fails renewal collection retries.

- **Action:** Downgrades user access.
- **Flow:**

1. Extracts the Stripe `customer` ID from the subscription object. 2. Queries the database for the user row matching `stripe_customer_id`. 3. Updates the user row:

  - Sets `is_pro = false`.

4. Logs the subscription termination.

Webhook Security & Logging

- **Service API client:** Webhook operations execute via the Supabase Service Role client (`createServiceClient()`) because webhook callbacks run outside user session contexts (unauthenticated requests).
- **Error Responses:**
- Returns `400` status for missing signatures or signature verification failures.
- Returns `500` status for DB update failures, instructing Stripe to retry sending the webhook event.